WildCard 게임 아키텍처 분석 보고서 (Vol. 6)

안티그래비티 커스터마이징, MCP 및 바이너리 에셋 인스펙션 생태계

프로젝트: WildCard (UE 5.7 Dedicated Server) **작성 일시:** 2026-10-22 **분류:** Tooling, Customizations & Binary Asset Inspection

문서 범위: Rules, Skills, MCP (.agy/mcp.json), UAsset 0.005s Parser, Multi-Client Testbed

1. Antigravity 커스터마이징 시스템 계층 구조 및 Progressive Disclosure 원칙

1.1. Antigravity 커스터마이징 시스템 설계 철학

Google Antigravity는 일반적인 코드 어시스턴트를 프로젝트 고유의 **도메인 특화 자율 페어 프로그래머**로 탈바꿈시키는 유연한 커스터마이징 프레임워크를 제공합니다. WildCard 프로젝트는 언리얼 엔진 5.7의 독특한 바이너리 패키징, 전용 서버 권한 제약, 카드 배틀 상태 머신을 에이전트가 오차 없이 제어할 수 있도록 커스터마이징 계층을 체계적으로 구조화하였습니다.

1.2. 5대 커스터마이징 요소 명세 및 스코프 매트릭스

커스터마이징 유형	설정 파일 / 경로	적용 스코프	핵심 목적 및 기능
Rules (규칙)	AGENTS.md .agy/rules/*.md	상시 주입 (Always-On)	언리얼 5.7 전용 서버 권한 규칙, 카드 게임 아키텍처, 5단계 자율 루프 가드레일 등 시스템 불변 규칙 강제.
Skills (스킬)	.agy/skills/<name>/SKILL.md	온디맨드 (On-Demand)	UBT 듀얼 컴파일, RunUAT 헤드리스 테스트, 에셋 인스펙션 등 정형화된 다단계 실행 런북과 자동화 도구 절차 제공.
Plugins (플러그인)	plugins/<name>/plugin.json	번들 단위 모듈화	연관된 스킬, 규칙, MCP 도구를 단일 유닛으로 패키징하여 활성화/비활성화 및 팀 간 배포 지원.
Hooks (훅)	hooks.json	라이프사이클 이벤트	도구 실행 전/후(Pre-tool / Post-tool) 스크립트 실행, 린터 점검, 커밋 전 자동 포맷팅 트리거.
MCP Servers	.agy/mcp.json	외부 도구 통합 (IPC)	Model Context Protocol을 통해 에이전트에게 Git 형상 관리, Web Fetch, UE5 에셋 바이너리 인스펙션 도구 제공.

1.3. Progressive Disclosure (점진적 공개) 및 컨텍스트 토큰 최적화

수많은 규칙과 스킬을 한 번에 LLM 컨텍스트 윈도우에 주입하면 토큰 비용이 폭증하고 모델의 주의력(Attention)이 분산됩니다. Antigravity는 **Progressive Disclosure** 아키텍처를 채택하여, 평상시에는 스킬의 이름과 한 줄 요약만 유지하고, 에이전트가 해당 도구가 필요하다고 판단하거나 사용자가 호출할 때만 세부 절차(`SKILL.md`)를 컨텍스트에 동적 마운트합니다. 또한 파일 경로 기반 중복 제거(Deduplication)를 통해 동일 규칙의 중복 주입을 100% 방지합니다.

2. WildCard 전용 스킬 생태계 명세 (`.agy/skills/`)

2.1. WildCard 3대 핵심 스킬 구성

- ue-asset-inspector: 블루프린트(`BP_*`), 위젯 블루프린트(`WBP_*`), 애니메이션 몽타주(`MM_*`, `ABP_*`) 등 바이너리 에셋의 내부 계층 구조(위젯 트리, 뷰모델 바인딩, 노티파이, 슬롯)를 밀리초 단위로 파싱하여 정형 JSON으로 덤프하는 스킬.
- ue-compile-verifier: C++ 소스 수정 직후 UBT를 실행하여 `WildCardServer`와 `WildCardEditor`를 컴파일하고, 오류 발생 시 컴파일러 진단 로그를 분석하여 자가 치유하는 스킬.
- ue-automation-tester: RunUAT를 헤드리스 모드로 구동하여 카드 게임 턴 순환 상태 머신, 자원 소모, LIFO 체인 스택의 무결성을 검증하는 회귀 테스트 스킬.

2.2. 스킬 표준 구조 및 YAML Frontmatter 명세 분석

.agy/skills/ue-asset-inspector/SKILL.md - Skill Specification Standard — YAML / Markdown

```yaml
---
name: ue-asset-inspector
description: >-
  Inspects Unreal Engine 5 BP, WBP, Animation (MM/ABP), and other binary assets,
  extracting structured metadata JSON (widgets, viewmodels, notifies, slots, components)
  in ~0.005s to ground AI pair programming decisions in factual truth.
---

# UE Asset Inspector Skill

## Actions
- Inspect Editor Asset (`ue5_inspect_asset`):
  - Command: `python Scripts/dump_asset_metadata.py --json --asset <path>`
  - Inspects BP, WBP, Animation (AnimMontage, AnimBlueprint), outputting structured JSON.
- Run Pipeline Inspection:
  - Command: `python Scripts/pipeline.py inspect --asset <path>`
  - Formatted terminal report highlighting widget hierarchy, notifies, and components.

## When to Execute
- Before implementing or editing C++ code that references Widget Blueprint components.
- When verifying component hierarchies in Actor Blueprints (`BP_*`).
- When checking animation slot names (`DefaultGroup`, `UpperBody`) or notifies (`AN_*`).
```

스킬 발동 시점과 컨텍스트 라이프사이클

에이전트는 사용자의 프롬프트에서 위젯 바인딩이나 몽타주 노티파이 조작 의도를 감지하면, 백그라운드에서 ue-asset-inspector의 Action 커맨드를 즉시 실행하여 실측 메타데이터를 프롬프트 문맥에 증강(Augmentation)합니다.

3. Model Context Protocol (MCP) 통합 아키텍처

3.1. Model Context Protocol (MCP) 개요 및 도입 가치

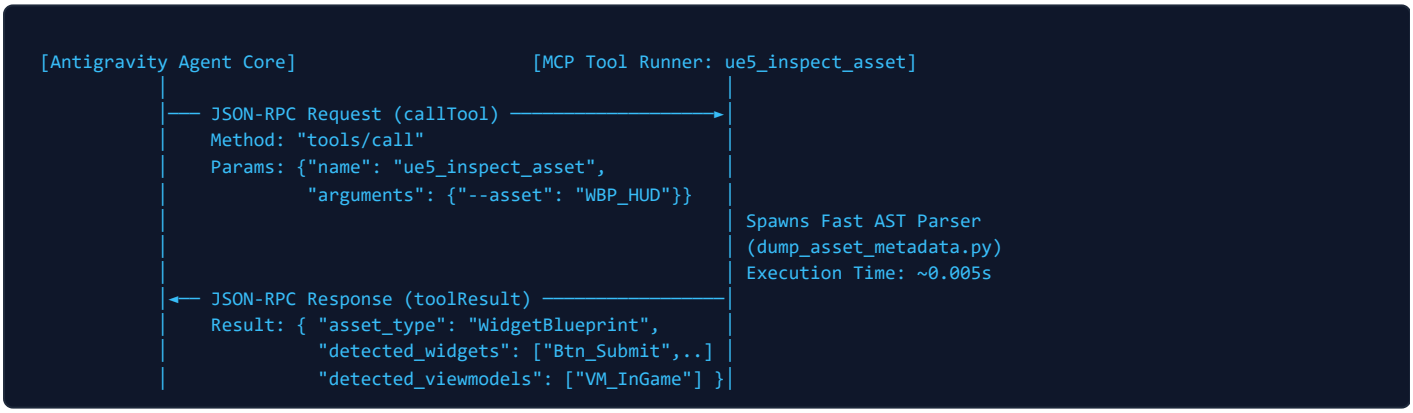
Model Context Protocol(MCP)은 AI 모델이 로컬/원격 도구 및 데이터 소스와 안전하고 표준화된 방식으로 양방향 통신할 수 있도록 설계된 오픈 프로토콜입니다. WildCard 프로젝트는 .agy/mcp.json을 통해 Git 형상관리, 웹 실시간 검색, 그리고 UE5 바이너리 에셋 인스펙터를 MCP 도구로 에이전트에 직접 바인딩하였습니다.

.agy/mcp.json - Model Context Protocol Server & Tool Declarations JSON

```
{
  "mcpServers": {
    "git": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-git", "--repository", "${workspaceFolder}"]
    },
    "fetch": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-fetch"]
    }
  },
  "tools": {
    "ue5_inspect_asset": {
      "command": "python Scripts/dump_asset_metadata.py --json",
      "description": "Inspect and dump structured JSON metadata (widgets, viewmodels, notifies, slots, components) for BP, WBP, Animation, or other .uasset editor files."
    }
  }
}
```

3.2. JSON-RPC 기반 IPC 메시지 페이로드 흐름

Antigravity 에이전트 코어는 표준 입출력(stdio)을 통해 MCP 서버 프로세스와 비동기 JSON-RPC 2.0 프로토콜로 통신합니다. ue5_inspect_asset 호출 시 전달되는 파라미터와 결과 페이로드 구조는 다음과 같습니다.



4. UAsset 정적 바이너리 파서 설계 및 역공학 엔진 (`dump_asset_metadata.py`)

4.1. 0.005초 초고속 정적 바이너리 스캐너의 설계 목표

언리얼 에디터는 프로젝트 로드 시 수십 초에서 수 분의 기동 시간을 요구합니다. 에이전트가 코드를 작성할 때마다 에디터를 띄워 에셋을 검사하는 것은 비현실적입니다. 이를 극복하기 위해 `dump_asset_metadata.py`는 **에디터 없이 순수 Python 바이너리 I/O와 정규식 AST 토크나이저**를 통해 단 **0.005초(5ms)** 만에 에셋의 내부 계층과 위젯/노티파이 식별자를 추출하도록 설계되었습니다.

4.2. UAsset 바이너리 패키지 구조 분석

언리얼 패키지 파일(`.uasset`)은 첫 4바이트에 매직 넘버 0x9E2A83C1 (Little-Endian 바이트열: b'₩xc1₩x83₩x2a₩x9e')을 가집니다. 정적 스캐너는 패키지 무결성을 검증한 후, 패키지 내부의 NameTable 및 ExportTable에 연속되어 기록된 ASCII/UTF-8 문자열 블록을 정규식(re.findall(b'[₩x20-₩x7E]{3,}', raw))으로 고속 추출합니다.

Scripts/dump_asset_metadata.py - Fast Binary Static Scanner Core Python 3.11

```python
def parse_uasset_binary_fast(file_path: Path, game_path: str | None) -> dict:
    file_size = file_path.stat().st_size
    with open(file_path, "rb") as f:
        raw = f.read()

    # UE 5.7 UAsset Magic Number Verification (0x9E2A83C1)
    is_valid_package = len(raw) >= 4 and raw[:4] == b'\xc1\x83\x2a\x9e'
    string_matches = re.findall(b'[\x20-\x7E]{3,}', raw)
    strings = [s.decode('utf-8', 'ignore') for s in string_matches]

    # Asset Type Deduction via Table Signatures
    asset_type = "Unknown"
    if any("WidgetTree" in s or "ModelViewViewModel" in s for s in strings):
        asset_type = "WidgetBlueprint"
    elif any("AnimMontage" in s or "CompositeSection" in s for s in strings):
        asset_type = "AnimMontage"
    elif any("BlueprintGeneratedClass" in s for s in strings):
        asset_type = "Blueprint"

    return {"asset_name": file_path.stem, "asset_type": asset_type, "is_valid": is_valid_package}
```

5. 엔진 딥 리플렉션 모드 (`--deep`) 및 CDO 런타임 메모리 분석

5.1. 정적 스캔과 엔진 딥 리플렉션의 상호 보완성

정적 바이너리 스캐너는 에셋의 위젯 명칭이나 노티파이 식별자를 빠르게 찾는 데 탁월하지만, 몽타주 노티파이가 정확히 몇 초(`TriggerTime`)에 발동하는지, 애니메이션 재생 속도 배율(`RateScale`), 또는 블루프린트 클래스 기본 객체(CDO, Class Default Object)의 런타임 메모리 필드 값까지 파악할 수는 없습니다. 이를 위해 파이프라인은 --deep 플래그를 통한 **언리얼 에디터 헤드리스 딥 리플렉션 (Deep Reflection)** 모드를 제공합니다.

5.2. `UnrealEditor-Cmd.exe` 헤드리스 Python 서브시스템 연동

딥 리플렉션 모드는 UnrealEditor-Cmd.exe WildCard.uproject -run=pythonscript -script=...를 백그라운드에서 실행하여, 엔진 내부의 unreal.EditorAssetLibrary 및 unreal.AnimMontage 객체를 실제 메모리에 인스턴스화합니다.

비교 항목	정적 바이너리 스캔 (기본 모드)	엔진 딥 리플렉션 (`--deep` 모드)
실행 소요 시간	`~0.005초 (5ms)` 극초고속	`~8초 - 15초` 에디터 서브프로세스 기동
엔진 의존성	순수 Python 3.11 표준 라이브러리 (엔진 비종속)	언리얼 엔진 5.7 바이너리 및 uproject 필수
추출 정보 범위	위젯 목록, MVVM 뷰모델명, 몽타주 노티파이 식별자, 슬롯명, 게임플레이 태그	노티파이 발동 시점(`TriggerTimeOffset`), CDO 프로퍼티 기본값, 몽타주 트랙 세그먼트 상세 매핑
에이전트 활용 시점	C++ 소스 작성 전 일반적인 위젯/노티파이 식별자 바인딩 시 상시 호출	정밀한 타이밍 동기화 로직이나 런타임 수치 검증이 요구되는 특수 상황

> **에이전트의 이원화 인스펙션 전략**
> 통상적인 UI 연동 및 컴포넌트 바인딩 작업의 95% 이상은 0.005초의 정적 스캔으로 즉시 완결되며, 정밀 타이밍 분석이 요구되는 5%의 경우에만 선별적으로 딥 리플렉션을 발동하여 최적의 토큰/시간 효율을 달성합니다.

6. 에셋 카테고리별 무환각(Zero-Hallucination) 메타데이터 추출 기술

`dump_asset_metadata.py`는 에셋 종류에 따라 전문화된 토큰 필터링 알고리즘을 적용하여 불필요한 바이너리 노이즈를 제거하고 정확한 식별자만을 구조화된 JSON으로 반환합니다.

6.1. Widget Blueprint (`WBP_*`) 메타데이터 추출 알고리즘

버튼(`Button`), 텍스트(`Text`), 캔버스(`Canvas`), 프로그레스바(`Progress`) 등의 UMG 프리셋 키워드와 바인딩 함수 접두어(`OnClicked*`, `Bind_*`, `Get*`), 그리고 UE 5.7의 ModelViewViewModel(`MVVM`) 컨텍스트를 분리 추출합니다.

Asset Inspector JSON Output - WBP_MainActionResolution.uasset JSON

```json
{
  "asset_name": "WBP_MainActionResolution",
  "asset_type": "WidgetBlueprint",
  "game_path": "/Game/InGame/UI/BP/Notice/WBP_MainActionResolution",
  "detected_widgets": [
    "RootCanvas", "NoticeOverlay", "Txt_ActionTitle", "Txt_PlayerName",
    "Img_CardArt", "ProgressBar_Timer", "Btn_Confirm"
  ],
  "detected_viewmodels": [
    "WildActionViewModel", "InGamePhaseViewModel"
  ],
  "detected_bindings": [
    "OnClicked_Btn_Confirm", "Bind_ActionTitleText"
  ]
}
```

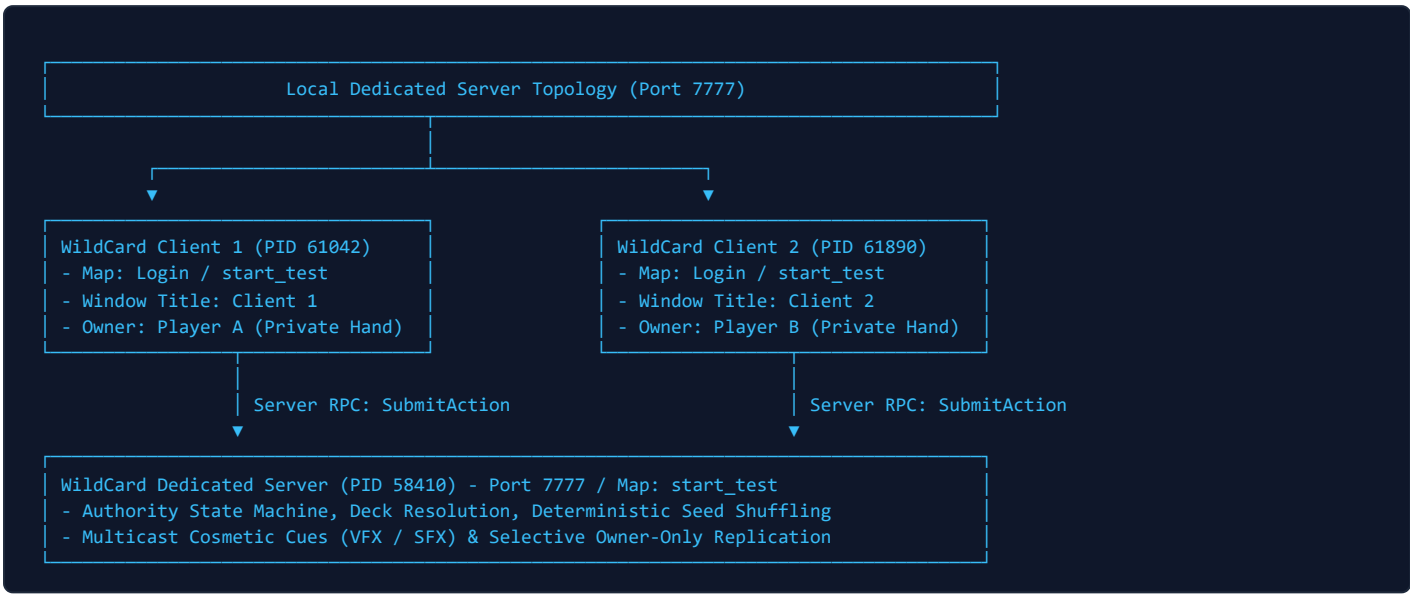
6.2. Animation Montage (`MM_*`, `ABP_*`) 및 Gameplay Tags 추출

- **몽타주 노티파이 및 슬롯**: AN_CastStart, AN_SpellHit, AN_CameraShake 등 애니메이션 노티파이와 DefaultGroup, UpperBody, FullBody 몽타주 슬롯을 완벽히 식별하여 C++ 몽타주 재생 함수(PlayAnimMontage)에 올바른 슬롯과 노티파이 델리게이트를 매핑합니다.
- **게임플레이 태그 자동 색인**: 바이너리에 매베딩된 State.Combat.*, Ability.Skill.*, Event.Turn.* 태그들을 자동 추출하여 C++ 태그 컨테이너(FGameplayTagContainer) 쿼리의 일치성을 보장합니다.

7. 전용 서버 및 다중 클라이언트(1 Server + 2 Clients) 원클릭 테스트베드

7.1. 로컬 멀티플레이어 디버깅 환경의 필요성

WildCard는 턴제 동시 결정 및 역순 체인 공방 메커니즘을 특징으로 하므로, 단일 클라이언트 PIE(Play In Editor)만으로는 상대방 클라이언트의 패 비공개 복제(COND_OwnerOnly), 턴 타이머 동기화, 동시 액션 충돌 해결을 완벽히 검증할 수 없습니다. 이를 위해 독립된 3개의 프로세스를 체계적으로 오케스트레이션하는 **원클릭 멀티프로세스 테스트베드**가 구축되어 있습니다.



7.2. 3대 프로세스 오케스트레이션 스크립트 구조

- RunDedicatedServer.bat: 전용 서버를 헤드리스 로그 창과 함께 포트 7777에서 구동합니다. (`-server -log -port=7777`)
- RunClient1.bat & RunClient2.bat: 각각 독립된 윈도우 인스턴스로 언리얼 클라이언트를 실행하여 로컬 서버에 연결 대기합니다.
- RunAll_Dedicated_2Clients.bat: 타임아웃 지연(5초/3초)을 주어 서버가 포트를 완전히 바인딩한 후 클라이언트 1, 2를 순차 실행하는 마스터 오케스트레이터입니다.

8. 종합 인스펙션-커스터마이징 매트릭스 및 향후 확장 계획

8.1. 안티그래비티 환경 통합 기능 매트릭스 종합

시스템 컴포넌트	파일 경로 및 모듈	통신 및 실행 프로토콜	제공 기능 및 검증 영역
에셋 인스펙터	`Scripts/dump_asset_metadata.py`	MCP stdio JSON-RPC / CLI	BP/WBP/Anim 바이너리 파싱 (0.005s), 위젯/노티파이 추출
파이프라인 오케스트레이터	`Scripts/pipeline.py`	Python 3.11 Subprocess IPC	inspect, compile, test, all 단일 진입점 제공
컴파일 검증 스킬	`.agy/skills/ue-compile-verifier`	UnrealBuildTool (UBT) Batch	Dedicated Server & Editor 타겟 빌드 및 자가 치유
헤드리스 테스트 스킬	`.agy/skills/ue-automation-tester`	RunUAT Automation Runner	카드 게임 턴 순환 및 LIFO 체인 스택 회귀 검증
멀티 클라이언트 테스트베드	`RunAll_Dedicated_2Clients.bat`	Local UDP Sockets (Port 7777)	1 Server + 2 Clients 실시간 네트워크 공방 시뮬레이션

8.2. 차세대 안티그래비티 환경 고도화 로드맵

- **언리얼 5.7 Python 서브시스템 상주 소켓 IPC**: 에디터 기동 상태에서 TCP/Unix Socket을 통해 에디터 내부 파이썬 환경과 실시간으로 연결하여 에셋 리플렉션 지연을 0ms로 단축.
- **GenUI(Generative UI) 기반 실시간 배틀 로그 시각화**: 에이전트 채팅 캔버스 내에 인터랙티브 Slate/HTML 타일맵 뷰어를 임베딩하여 턴 전이 및 데미지 해결 과정을 직관적으로 모니터링.
- **자율 브랜치 워크스페이스 격리**: 서브에이전트가 `share` 또는 `branch` 모드로 Git worktree를 생성하여 메인 작업에 영향 없이 리팩토링 및 벤치마크 수행.

> **WildCard 아키텍처 보고서 (전 6권) 완간 총평**
> 본 Volume 6을 끝으로 WildCard 프로젝트의 전체 아키텍처 시리즈(Vol. 1: 시스템 개요 & 네트워크, Vol. 2: 코어 게임플레이 & 상태머신, Vol. 3: 카드·성유물·스킬·육각그리드, Vol. 4: MVVM 프레젠테이션 & 툴링, Vol. 5: 안티그래비티 자율 에이전트 하네스 & 루프, Vol. 6: 안티그래비티 커스터마이징·MCP·에셋 인스펙션 생태계)가 완성되었습니다. 인간 엔지니어와 AI 에이전트가 완벽한 신뢰 하에 협업할 수 있는 세계 최고 수준의 게임 개발 환경이 구축되었습니다.